Course Pipeline Audit — NexoraXR

Scope: upload → ingestion → RAG → planning → script generation → script review → TTS. Evidence source: live Postgres (`classrooms`, `courses`, `course_documents`, `course_sections`, `course_chunks`, `course_lectures`, `course_builds`), `profbrain/lectures/*` artifacts, `nexora_ai/lecture_cache/*` mtimes, code inspection. No secrets included.

---

1. Stage-by-stage pipeline

Stage	Code	Behavior	Models / config
Upload	`server.js` `POST /api/rooms/:roomId/materials`	Host-only; ≤8 PDFs, ≤25 MB each; registers `course_documents`, ingests async	—
Ingestion	`courses/pipeline.mjs` `parsePdfPages` / `ingestDocument`	`pdf-parse` **pagerender** (pdf.js textContent order) → per-page text → `parseSections` (heading heuristic, `<3` sections → one-per-page fallback) → `chunkSection` (≤700-word chunks, 120-word overlap, never crosses pages) → `course_chunks` + `course_sections`; per-doc LLM summary+concepts (≤12 sections)	Ollama `chat` for summaries; `embed` for chunk embeddings
Planning	`generator.mjs` `buildOutline`	Reads doc **summaries + concepts** (not raw chunks) → exactly-N-lecture outline, sources with page ranges	Ollama chat
Retrieval	`generator.mjs` `retrieveChunks`	Embedding cosine over `course_chunks` (k = `RETRIEVE_K`, over-fetched ×3, scoped to assigned doc/page ranges; per-doc cap 60%; `isAdminText` filter drops logistics chunks)	Ollama embed
Writing	`generator.mjs` `writeSegments` + `parseSegments` + `validateLecture`	Prompt: 8–12 segments, ~`TARGET_WORDS`, JSON schema with section/document/page; up to `MAX_WRITE_ATTEMPTS` with targeted retry hints; validation: structure, word band, page provenance, no admin narration	Ollama chat
Script review	`server.js` `GET/PUT /api/courses/:id/drafts`, `POST`	Build stops at `awaiting_review`. Reviewer	none (deterministic)

Stage	Code	Behavior	Models / config
	`.../approve-audio`; `courses/drafts.mjs`	edits per-segment **or** pastes a whole script → `splitScriptIntoSegments` + `alignSegments` (kept/auto/uncertain page mapping) → artifacts rewritten → `validateDraftScript` → atomic `awaiting_review→audio` flip (dup-click safe)	
Audio (TTS)	`courses/bake.mjs` `bakeLecture`/`verifyCourseAudio` → `lectureRoutes.mjs` `generateSegmentAudio` → `voxcpmClient.mjs` `synthesize`/`synthesizeVerified`	Per lang: manifest (per-segment translation for non-en) → per-segment WAV (concurrency `COURSE_TTS_CONCURRENCY=2`) → `warmLectureUtterances` (opening/transitions/closing) → slide-image warm (en)	VoxCPM2 via vLLM-Omni `/v1/audio/speech`; Whisper `detectLanguagesWindowed` for non-en verification

Delivered build statuses

`ingesting → planning → scripts → awaiting_review → audio → validating → ready | failed`
(Builds parked in `awaiting_review` are excluded from crash-resume; `audio`/`validating` resume mid-stage.)

---

2. Haptics build — where the 11 sections disappear

File facts (evidence):

- Stored PDF `uploads/materials/priv_7ec3ced3155e731339/Haptics_Course_Primer.pdf`, md5 `cc8d358a…`, 2 pages, **two columns**. Both `pdftotext` (raw) and `pdftotext -layout` were compared: raw mode scrambles column order (`§1, §2, §5, §3, §4, §6…`), layout mode is clean. **pdf-parse is used, and it preserved reading order** — the scramble is not the pipeline's bug.
- `course_sections`: 1 row ("Document start", p.1–2) — numbered headings (`1. What is haptics?`) aren't caught by the heading heuristic, but no text is lost at this stage.
- `course_chunks`: **3 chunks — 4633 + 1920 + 4274 chars** — all 11 sections present (chunk 0 ≈ §1–4, chunk 1 ≈ §4–6, chunk 2 ≈ §7–11).
- Retrieved chunks: all 3 pass scope (single doc, pages 1–2 assigned).

The loss point — `generator.mjs:289`:

```
`[${i+1}] ${c.filename} page ${c.page} — ${c.heading}\n${c.text.slice(0, 1500)}`
```

Each chunk is truncated to **1500 chars** in the writer context:

- chunk 0 → only ~32% sent (§3–§4 head cut)
- chunk 1 → ~78% sent (mostly intact)
- chunk 2 → ~35% sent (§9–§11 cut)

Result (`profbrain/lectures/gen_79165755_L1/segments.json`): 8 segments, **190 words total**, all `p.1`, sections = Deformable Objects → Teleoperation Architectures — i.e. chunk 1 + head of chunk 2. §1–3 were inside the context but

the model's 8-segment budget skipped them (LLM allocation, not pipeline loss).

Validation did flag it: `artifacts.validation.ok=false`, `duration 2.9m outside 4.5–5.5m band` — the review gate is holding this lecture back correctly.

Confirmed bug: `writeSegments` per-chunk `slice(0,1500)` silently discards tail sections of long chunks. **Not bugs:** extraction, chunking, retrieval, review API, editor rendering — all preserve the content.

---

3. TTS delivery — measured vs hypothesized

Delivery mode (verified): `POST {VOXCPM_BASE_URL}/v1/audio/speech`, `stream:false`, `response_format:'wav'` — a **complete WAV is returned per call; nothing is playable before the request finishes**. Voice conditioning via `ref_audio`/`ref_text` from `voxcpm_data/voice_registry.json` (per-lang entries `<voice>_<lang>` preferred).

Languages prepared: `COURSE_BAKE_LANGS` **is read** (`bake.mjs:145`, dotenv loaded before import) — default `en,ar,fr,de,es,zh`, sequential (`verifyCourseAudio` lang loop). Within a language, segment TTS runs 2-wide (`COURSE_TTS_CONCURRENCY`, default 2). Per-segment lazy runtime path also exists (`/lecture/segment_audio` generates on first play).

Caching: `lecture_cache/segments/<lid>[__<lang>]/segment_NNN.{wav,txt,_lipsync.json}`; `generateSegmentAudio` returns early when the wav exists; drafts-save deletes changed segments' cache in **all** language buckets.

Retries — nested amplification (code-verified):

- `synthesizeVerified`: up to **3** (synth + Whisper window-detect) attempts for non-en; en skips detection.
- `generateSegmentAudio` wraps that in **another 3** attempts with 2s/4s/6s backoff.
- **Worst case per non-en segment: 9 TTS + 9 Whisper calls.**

Validation: `verifyCourseAudio` measures real WAV durations (`wavDurationMs`); en must land in `AUDIO_MIN/MAX_S` (270–330s) or the build fails; partial language coverage is non-fatal and recorded in `artifacts.baked_langs`/`partial_langs`.

Measured timings (`gen_79165755_L1` cache dir mtimes — smallest case: 8 segments, 190 words):

- en dir created 20:09:24; lang dirs appear sequentially — ar 20:14:26, fr 20:16:04 (~1.6m gap), de 20:19:41 (~3.6m), es 20:24:03 (~4.4m); zh absent. Total ≥16 min.
- Conclusion (evidence): languages are strictly serial; per-language cost ~2–4 min even for a tiny lecture.

Hypotheses (not measured): per-segment Ollama translation inside `loadLectureManifest` (serial loop, `lectureRoutes.mjs:344`) and Whisper verification likely dominate; individual VoxCPM calls appear to be seconds each. Instrumentation needed to split translate/TTS/detect/queue.

---

4. Confirmed bugs

1. **`writeSegments` truncates every chunk to 1500 chars** (`generator.mjs:289`) → tail sections of long chunks never reach the writer. *Root cause of Haptics coverage loss.*
2. **Nested retries amplify worst case 9×** (`synthesizeVerified` inside `generateSegmentAudio`'s own 3-attempt loop).
3. **All languages baked serially before `ready`** — build latency scales linearly with `BAKE_LANGS`.
4. (Cosmetic) `parseSections` doesn't recognize `N. Title` numbering → multi-section docs collapse to "Document start" (text survives; only section granularity lost).

5. Unresolved questions

- Per-call cost split translate vs TTS vs Whisper vs utterance-warm — needs timing instrumentation.
- Whether `MAX_WRITE_ATTEMPTS` retries re-truncate context identically (likely yes — same `chunks` list is reused).
- Whether review-edited scripts should re-run `validateLecture` provenance checks (currently `validateDraftScript` only checks size/emptiness; page mappings flagged `uncertain` are advisory).

6. Prioritized fixes

Coverage

1. Raise `writeSegments` chunk cap (1500 → ~4000) or chunk-text-aware inclusion — one-line change, restores missing sections.
2. Improve `parseSections` heading regex to accept `^\d+\.\s+` short titles (already half-supported; widen the `<3 sections` fallback).

Latency 3. Bake only the instructor-selected language before `ready` (`build.lang` already stored); report per-language `baked/preparing/unprepared` status so switching never silently stalls — lazy generation already exists, it needs status surfacing. 4. Remove the outer retry wrap when `synthesizeVerified` already retries (9 → 3 worst case). 5. Optional: parallelize `BAKE_LANGS` loop or raise `COURSE_TTS_CONCURRENCY` (bounded — same local GPU).